Design Twitter

Problem Statement: Create a simplified version of a social media platform similar to Twitter. Users should be able to post tweets, follow or unfollow other users, and view the 10 most recent tweets in their news feed.

Implement the Twitter class as follows:

- **Twitter()**: Initializes the Twitter object.
- **void postTweet(int userId, int tweetId)**: Composes a new tweet with ID `tweetId` by the user `userId`. All `tweetId`s will be unique.
- **List<Integer> getNewsFeed(int userId)**: Retrieves the 10 most recent tweet IDs in the user's news feed. The news feed should only show posts from users the user follows or from the user themself, with tweets arranged from most recent to least recent.
- **void follow(int followerId, int followeeId)**: The user with ID `followerId` started following the user with ID `followeeId`. Input will be given such that `followerId` is not already following `followeeId` at the time of function call.
- **void unfollow(int followerId, int followeeId)**: The user with ID `followerId` unfollowed the user with ID `followeeId`. Input will be given such that `followerId` is following `followeeId` at the time of function call.

Examples

```
Input: [postTweet(1, 2), postTweet(2, 6), getNewsFeed(1), follow(1, 2),
getNewsFeed(1), unfollow(1, 2), postTweet(1,7), getNewsFeed(1)]
Output: [null, null, [2], null, [6, 2], null, [7, 2]]
Explanation:postTweet(1, 2): User with userId 1 posts tweet with tweetId 2.
postTweet(2, 6): User with userId 2 posts tweet with tweetId 6.
getNewsFeed(1): Outputs feed of userId 1 = [2].
follow(1, 2): User with userId 1 follows user with userId 2.
getNewsFeed(1): Outputs feed of userId 1 = [6, 2], it contains the news of
userId 2 as well.
unfollow(1, 2): User with userId 1 unfollows user with userId 2.
postTweet(1,7): User with userId 1 posts tweet with tweetId 7.
getNewsFeed(1): Outputs feed of userId 1 = [7, 2].

Input: [follow(2, 1), follow(1, 2), postTweet(1, 6), getNewsFeed(1),
getNewsFeed(2), unfollow(2, 1), getNewsFeed(2)]
Output:  [null, null, null, [6], [6], null, []]
Explanation:follow(2, 1): User with userId 2 follows user with userId 1.
follow(1, 2): User with userId 1 follows user with userId 2.
postTweet(1, 6): User with userId 1 posts tweet with tweetId 6.
getNewsFeed(1): Outputs feed of userId 1 = [6].
getNewsFeed(2): Outputs feed of userId 2 = [6]. This includes tweets of userId 1
as well.
unfollow(2, 1): User with userId 2 unfollows user with userId 1.
getNewsFeed(2): Outputs feed of userId 2 = [], its empty as userId 2 no longer
follows userId 1
```

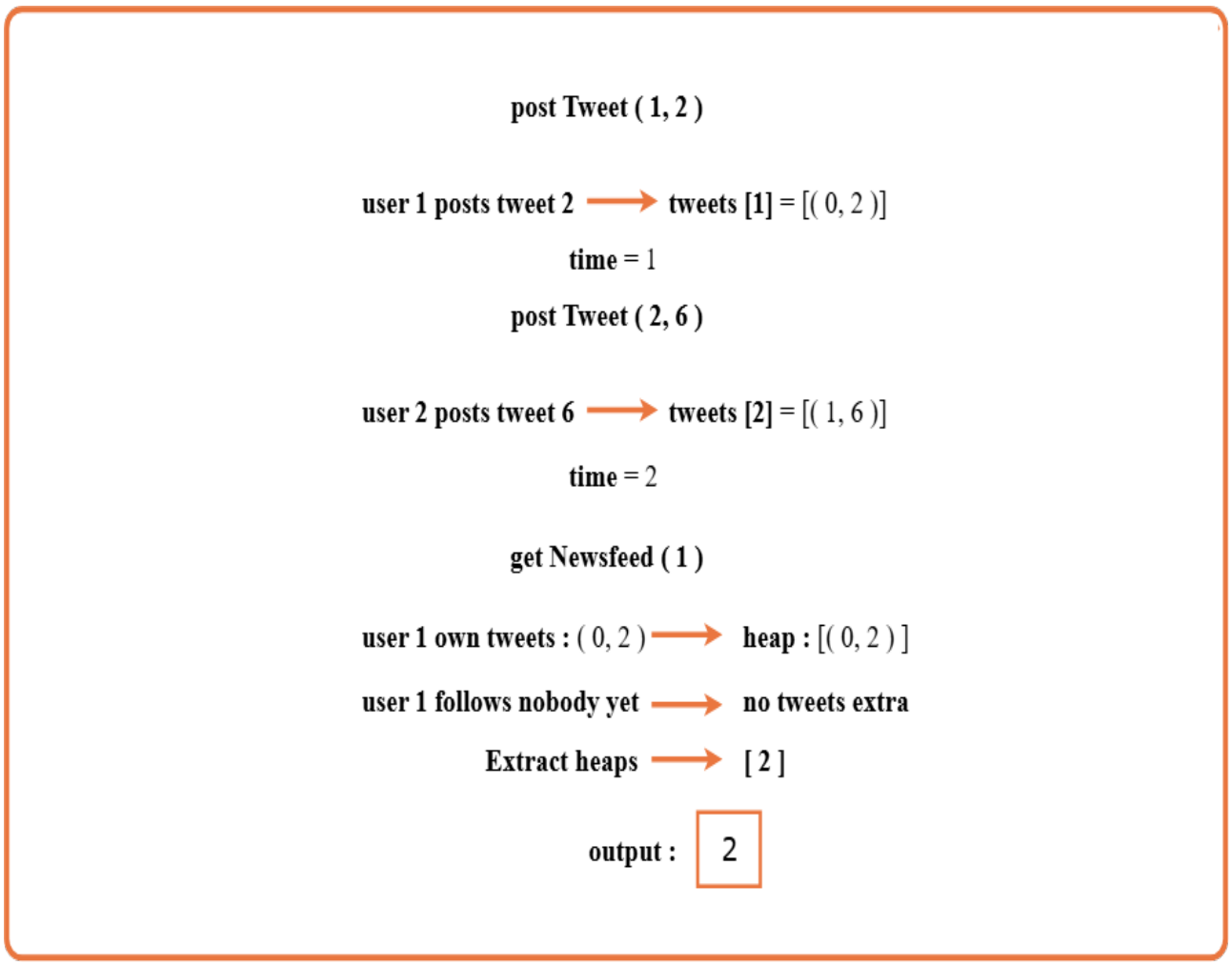
Approach

Algorithm

The main challenge in this problem is retrieving the 10 most recent tweets across all followed users in an efficient way. Since each user may have multiple tweets, and tweets are ordered by recency, this becomes similar to merging K sorted lists (one per user). To do this optimally, we use a Max-Heap (priority queue) to always fetch the most recent tweet. The heap keeps at most one tweet from

each list (user), and we simulate a merge sort–like process where we pop the latest tweet and push the next most recent tweet from the same user (if any). This ensures that we only process what's necessary instead of sorting everything or traversing all tweets.

- Maintain a map from each user to a list of their tweets (each tweet stores its timestamp).
- Maintain a map of user to set of users they follow.
- In `postTweet()`, add the tweet with a timestamp to the user's list.
- In `follow()` and `unfollow()`, update the user's follow set accordingly.
- In `getNewsFeed()`, use a max heap to store the most recent tweets across all followed users and self.
- For each user being followed (including self), if they have tweets, push their most recent tweet into the heap along with an index pointer.
- Pop from the heap at most 10 times, each time adding that tweet ID to the result.
- If more tweets exist for that user, push the next recent tweet from their list into the heap.
- This simulates merging sorted tweet streams efficiently.



follow (1, 2)

user 1 follows user 2 → **following [1] = { 2 }**

get Newsfeed (1)

user 1 own tweets : (0, 2) → **heap :** [(0, 2)]

tweets from followee 2 : (1, 6) → **push** →

heap : [(0, 2), (1, 6)]

heap size ≤ **10 (keep both)**

Extract heap in reverse → **[6, 2]**

output :

6	2

unfollow (1, 2)

user 1 unfollows user 2 → **following [1] = { }**

post Tweet (1, 7)

user 1 posts tweet 7 → **tweets [1]** = [(0, 2), (2, 7)]

time = 3

get Newsfeed (1)

user 1 own tweets : (0, 2), (2, 7) → **heap :** [(0, 2), (2, 7)]

user 1 follows nobody yet → **no tweets extra**

extract heap in reverse → **[7, 2]**

output :

7	2

Code

```
import java.util.*;

// Class representing the Twitter system
class Solution {
    // Map to store user's tweets (timestamp, tweetId)
    Map<Integer, List<int[]>> tweets;
    // Map to store who follows whom
    Map<Integer, Set<Integer>> following;
    // Time counter to give each tweet a unique timestamp
    int time;

    // Constructor to initialize the system
    public Solution() {
        tweets = new HashMap<>();
        following = new HashMap<>();
        time = 0;
    }

    // Function to post a new tweet
    public void postTweet(int userId, int tweetId) {
        tweets.putIfAbsent(userId, new ArrayList<>());
        tweets.get(userId).add(new int[]{time++, tweetId});
    }

    // Function to retrieve 10 most recent tweet IDs in the news feed
    public List<Integer> getNewsFeed(int userId) {
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);

        // Add own tweets
        if (tweets.containsKey(userId)) {
            for (int[] tweet : tweets.get(userId)) {
                pq.offer(tweet);
                if (pq.size() > 10)
                    pq.poll();
            }
        }

        // Add tweets from followed users
        if (following.containsKey(userId)) {
            for (int followee : following.get(userId)) {
                if (tweets.containsKey(followee)) {
                    for (int[] tweet : tweets.get(followee)) {
                        pq.offer(tweet);
                        if (pq.size() > 10)
                            pq.poll();
                    }
                }
            }
        }

        // Retrieve tweets in reverse order
        LinkedList<Integer> res = new LinkedList<>();
        while (!pq.isEmpty()) {
            res.addFirst(pq.poll()[1]);
        }
        return res;
    }

    // Function to follow another user
    public void follow(int followerId, int followeeId) {
        following.putIfAbsent(followerId, new HashSet<>());
```

```
        following.get(followerId).add(followeeId);
    }

    // Function to unfollow another user
    public void unfollow(int followerId, int followeeId) {
        if (following.containsKey(followerId))
            following.get(followerId).remove(followeeId);
    }
}

// Driver class
class Main {
    public static void main(String[] args) {
        Solution twitter = new Solution();

        twitter.postTweet(1, 2);
        twitter.postTweet(2, 6);

        System.out.println(twitter.getNewsFeed(1)); // [2]
        twitter.follow(1, 2);
        System.out.println(twitter.getNewsFeed(1)); // [6,2]
        twitter.unfollow(1, 2);
        twitter.postTweet(1, 7);
        System.out.println(twitter.getNewsFeed(1)); // [7,2]
    }
}
```

Complexity Analysis

Time Complexity: postTweet, follow, and unfollow run in O(1). getNewsFeed takes O(log k) where k is the number of users followed (including self), since we maintain a min-heap with at most one tweet per user and extract up to 10 tweets.

Space Complexity: Storing tweets takes O(n), where n is total tweets. Follow relationships take $O(u^2)$ in worst case (if everyone follows everyone). Min-heap in getNewsFeed uses O(k) space, where k is number of users followed.